SignDesk

A digital document signing platform. Send a PDF by email, sign it in the browser, and receive a file sealed with a PAdES-B-LTA digital signature and a trusted timestamp.

Live demo	https://docsignass.faisalkhan.cloud/demo
Source	https://github.com/samfaiz/DocuSignAssignment
Demo login	demo@signdesk.test / demo-signdesk-2026
Prepared	August 2026

How to read this document

Sections 4, 5, 7, 8 and 9 answer the five questions set with the assignment: the tech stack, why that stack and not the alternatives, the competition and their pricing, the security case for building in-house, and the core concepts of digital signatures. The remaining sections describe what was actually built, which runs and is deployed.

Every claim about signature levels, trusted timestamps and tamper detection is reproducible, and is confirmed by pyHanko's command-line validator - a tool that is not part of this codebase. Section 12 shows that output.

1. The assignment

Digital Docu sign - Admin should be able to send an email with a PDF and a link to digitally sign the document on my portal, plus a normal signature feature like Adobe Acrobat: import an image (PNG signature), or write a name which automatically mimics a signature using a signature-friendly font.

Plan: (1) tech stack, (2) reason for selecting it and why not others, (3) competition with monthly costing, (4) security benefits of an in-house solution versus third party, (5) core concepts of digital signatures and what they capture.

2. What was built

- **Admin portal** - upload a PDF, click the page to place fields per recipient, send, and track status.
- **Signer portal** - tokenised email link, one-time passcode, consent, sign, done. Signers have no account.
- **Three signature modes** - draw it, upload a PNG or JPEG, or type a name rendered in one of five bundled script fonts.
- **Server-side stamping** - the browser never decides where ink lands in the final document.
- **Certificate of Completion** - identity method, consent version, document hashes and the full event timeline, appended to every signed document.
- **PAdES-B-LTA sealing** - PKCS#7 signature, RFC 3161 trusted timestamp, embedded revocation data and a renewable document-timestamp chain.
- **Hash-chained, append-only audit trail** enforced by database triggers.
- **Optional per envelope** - the signer's location and a photograph, both consented, both recorded as reported rather than verified.
- **Enforced retention** - photographs and coordinates deleted on a schedule.
- **Admin-managed SMTP** - mail configured from the interface, password encrypted at rest.
- **Public verification** - upload any PDF and check whether it has been altered since signing.

Trying it

Open the demo link above and press **Start signing ceremony**. It creates an agreement and drops you into the signer's shoes - no account, no terminal. The one-time passcode is shown on screen rather than emailed, so there is no inbox to hunt through. A **Guided tour** panel, off by default, follows whichever step you are on and explains the decision behind it.

Those conveniences are gated behind a demo flag that defaults to on only in a local environment. One of them reveals a live authentication factor, so the routes do not register anywhere else and the controller re-checks the flag. Turning the flag off removes them entirely.

3. The technical terms, in plain English

This section exists so the rest of the document can be read without a background in cryptography. Nothing here is simplified to the point of being wrong - each entry is the honest short version.

Signing and cryptography

Term	What it actually means
Hash, or SHA-256	A fingerprint of a file. Put in any amount of data, get back 64 characters. Change one letter anywhere and the fingerprint changes completely, and you cannot work backwards from it to the file.
Public and private key	A matched pair of numbers. What one locks, only the other unlocks. You keep the private one secret and hand out the public one.
Digital signature	Take the document's fingerprint, lock it with your private key, and attach the result. Anyone can re-fingerprint the document and unlock your attachment with your public key. If the two match, nothing has changed and the holder of that private key signed it.
Certificate	A file saying "this public key belongs to this person or organisation", signed by an authority. Like a passport - useful only because of who issued it.
Certificate authority (CA)	The body that issues certificates. Adobe and your operating system ship with a list of the ones they trust. A certificate from outside that list still works cryptographically, but shows as untrusted.
PKCS#7, or CMS	The standard envelope the signature is packed into: the locked fingerprint, the certificate, and a few details such as the claimed signing time.
PKCS#12, or a .p12 file	A password-protected file holding a certificate and its private key together. This is what the sealing service loads to sign with.
Timestamp authority, RFC 3161	An independent service that stamps the time onto your signature using its own clock. Needed because your own computer's clock proves nothing - you could set it to any date you like.
Revocation, CRL, OCSP	A certificate can be cancelled before its expiry date, for instance if the key is stolen. A CRL is the published list of cancelled ones. Checking that list is how a verifier knows the certificate was still good at the moment of signing.

How a signature lives inside a PDF

Term	What it actually means
ByteRange	A signature cannot sign itself - writing it in would change the very bytes it just fingerprinted. So the PDF leaves a gap for it, and the ByteRange records "everything except this gap is what was signed".
Incremental update	PDFs are added to rather than rewritten. Signing appends to the end of the file, which is why every earlier version of the document is still inside it and can be recovered.
PAdES	The rulebook for putting signatures into PDFs, so that every reader agrees on what a valid one looks like.
DSS dictionary	A pocket inside the PDF where the certificate chain and the revocation paperwork are stored, so a verifier does not have to fetch them from the internet years later.
Long-term validation (LTV)	The general idea of putting everything a future verifier needs inside the file itself, because websites and certificate authorities do not last forever.

What PAdES-B-LTA actually means

Four letters doing a lot of work. Each level adds one thing to the one before:

Level	Plain English	The question it answers
B-B	It is signed.	Has anyone changed this document since? Who held the signing key?
B-T	Plus an independent timestamp.	When was it signed, according to a clock nobody involved controls?
B-LT	Plus the paperwork to check the certificate.	Can this still be checked after the issuing authority shuts down?
B-LTA	Plus that paperwork gets refreshed over time.	Will this still verify in twenty years, after the certificate itself has expired?

So **PAdES-B-LTA** is the strongest of the four: a signed PDF, stamped by an independent clock, carrying everything a future reader needs to check it, in a form that can be topped up before the cryptography ages out. It is the level used for documents expected to matter for decades - and it is what this system produces.

Security terms used later

Term	What it actually means
Hash chain	Each entry in the audit log includes the fingerprint of the entry before it. Change entry five and every entry after it stops adding up - like a numbered ledger where each page quotes the previous page.
Append-only	Rows can be added but never edited or deleted. Enforced by the database itself here, not merely by the application.
bcrypt	A deliberately slow way of storing a password or passcode. Slowness is the feature: it makes guessing millions of possibilities impractical.
One-time passcode (OTP)	The six digits emailed to the signer, usable once and only for a few minutes.
HMAC	A shared secret used to prove a message came from who it claims and was not altered on the way. Used between the API and the sealing service.
IDOR	A common bug where changing an identifier in a URL shows you someone else's data. Prevented here by scoping every lookup to the person asking, rather than trusting the identifier.
Rate limiting	Capping how often something can be attempted, so automated guessing becomes impractical.
EXIF	Hidden data cameras write into photographs - the model, the time, and very often the GPS coordinates. Stripped here, because this system asks about location separately.

Everything else

Term	What it actually means
API	The set of web addresses the front end talks to. It returns data, not pages.
Single-page app (SPA)	The browser loads one page and rewrites parts of it as you navigate, instead of fetching a whole new page each time.
Bearer token	A random string that proves you are logged in, sent with every request.
Queue and worker	Slow jobs are handed to a background process so the person clicking is not left waiting. Sealing runs here, because it talks to a timestamp authority over the internet.
jsonb	PostgreSQL's way of storing flexible, nested data that is still searchable - used for the details attached to each audit entry.

Term	What it actually means
SMTP	The protocol for sending email. Configured from the admin screen rather than a config file, so mail can be fixed without shell access.
Envelope	The industry's word for one document sent out for signature, along with its recipients, fields and history. Borrowed from DocuSign.

4. Question one: tech stack

Layer	Choice
Admin and signer UI	React 19, TypeScript, Vite 8, Tailwind 4, React Router 7
PDF rendering in the browser	pdf.js 6
API	Laravel 13, PHP 8.3+, Sanctum
Database	PostgreSQL 16
Queue and cache	Redis 7
Object storage	S3-compatible, or local disk
Email	SMTP, configured from the admin interface
PAdES sealing service	**Python 3.12, FastAPI, pyHanko 0.36**
PDF composition	pypdf, reportlab, Pillow
Trusted timestamps	RFC 3161 - DigiCert
Certificates	OpenSSL CA publishing a CRL; a commercial CA or CCA-licensed ESP in production
Orchestration	Docker Compose locally; nginx, PHP-FPM and systemd in production
Tests	PHPUnit, Python integration scripts, end-to-end ceremony runner

Three processes run: a React single-page app, a Laravel API, and a Python sealing service the API calls over a private network with no public ingress.

5. Question two: why this stack, and why not the alternatives

Laravel for the API

The feature list is almost exactly Laravel's standard equipment: queues for the sealing pipeline, mail with an SES driver, an S3 storage abstraction, Sanctum tokens, authorisation policies, and a scheduler for retention and expiry sweeps. None of that had to be built. It is also the framework I have shipped production work in, so effort went into the signing logic rather than into learning a framework.

React for the interface

Two screens carry real interaction state: the field-placement editor, with click-to-place, zoom and per-page coordinate mapping, and the signature pad, with canvas drawing, image upload and live font preview. A component model with local state fits that far better than server-rendered templates with sprinkled interactivity.

A Python sealing service - necessity, not preference

This is the most important decision in the project, and it was forced. **PAdES B-LTA cannot be produced in PHP with free libraries.** Four options were evaluated before adding a second language:

Option	Verdict
FPDI (free)	Cannot read PDF 1.5+ at all. PDF 1.5 introduced compressed cross-reference streams and object streams; the open-source parser does not support them, and most real-world PDFs are 1.5 or later. Reading them requires the **commercial** FPDI PDF-Parser add-on.
TCPDF setSignature()	Produces only a basic adbe.pkcs7.detached signature - no PAdES subfilter, no DSS dictionary, no document-timestamp chain. Structurally incapable of exceeding B-B.

Option	Verdict
SetaPDF-Signer	Implements PAdES properly. Commercially licensed.
pyHanko (Python)	Open source, covers B-B, B-T, B-LT and **B-LTA** with full long-term validation.

So the choice was: buy a licence, ship a weaker signature while describing it as something it is not, or add roughly 200 lines of Python behind one internal HTTP call. Only the last delivers what was asked for, at no licence cost, with the cryptography handled by a library that specialises in it.

PostgreSQL rather than MySQL

The jsonb type with GIN indexing keeps audit payloads queryable without a second denormalised table, and CHECK constraints are expressive enough to enforce the envelope state machine and field-coordinate bounds in the database rather than only in application code.

One caveat found in practice: **jsonb does not preserve object key order**, returning keys sorted by length then bytewise. That broke the audit hash chain on its first real run, because a payload written in one order re-serialised in another and the recomputed hash no longer matched. The fix was to canonicalise - recursively sorting keys - before hashing, the same idea as RFC 8785.

PostgreSQL rather than MongoDB

The domain is inherently relational - envelope, recipient, field, value - and finalising a signature is a multi-table transaction. An evidence log needs strict ordering and referential integrity, which is the opposite of what eventual consistency offers.

Object storage rather than database blobs

PDFs run from 100 KB to 20 MB. Blobs bloat backups, slow replication and cannot be streamed. S3 also brings versioning, lifecycle rules, server-side encryption and pre-signed URLs. The disk is configurable, so a single-server deployment can use local storage with no code change.

Queues rather than synchronous sealing

Sealing makes a network round trip to a timestamp authority and fetches revocation data. It must be retryable with backoff, not blocking a request.

Why not Next.js

A good stack, but it would have meant learning a framework and implementing cryptography at the same time. The Python service is required regardless, so the single-language argument disappears.

Why not Node and Express

Queues, mail, storage and authentication would all have to be assembled by hand.

Why not sign in the browser with WebCrypto

The private key would have to reach the client. That is unacceptable key custody at any scale, and client-reported coordinates and timestamps cannot be trusted anyway. All sealing is server-side; the browser only ever produces a preview.

6. Working flow

Three stages. The first two are interactive; the third runs on a queue.

Stage one - the admin prepares and sends

Upload. The magic bytes are checked, then the file is handed to the Python service where a real parser confirms it opens, is not password-protected, and reports its page count. Only then is it stored under a random key - never the user's filename - and its SHA-256 recorded. That hash is the anchor everything downstream is compared against.

Field placement. Coordinates are stored as fractions of the page between 0 and 1 with a top-left origin, not pixels, so a placement survives zoom, screen DPI and the later conversion into PDF user space. Every field belongs to exactly one recipient, which is what makes the rule that nobody can fill another signer's field enforceable rather than merely hidden in the interface.

Send. Each recipient gets a 256-bit random token. Only its SHA-256 is stored, so a database dump yields no working links.

Stage two - the signer completes the ceremony

Signers have no account, so every request re-establishes who is calling from the token alone. The token is looked up by hash and compared in constant time.

Passcode. Six digits, hashed with bcrypt rather than SHA-256 - a million possibilities would fall to an offline sweep against a fast hash - expiring in ten minutes and locking out after five failures. The document is not served until this passes: possession of a forwarded link is not enough.

Consent. Records the disclosure version and a SHA-256 of the exact text shown, not a boolean. The question in a dispute is never whether someone consented but to what wording.

Optional evidence. If the sender enabled it, dialogs ask for the signer's location and a photograph. Both are declinable, both decisions are recorded, and neither blocks signing.

Signing. Typed names are rendered to PNG on the server, so the artefact sealed into the document does not depend on which fonts the signer happened to have installed. Uploaded images are fully decoded and re-encoded, which strips EXIF. Clicking the final button is logged as its own intent event, separate from having filled the fields in - that distinction is what the ESIGN Act and UETA actually turn on. The token is then burned.

Stage three - the server seals and delivers

Queued, with widening backoff, because sealing makes a network round trip that can fail for reasons nothing to do with the signer.

The job reads field coordinates from the database, never from the finishing request, so a signer cannot move their own signature elsewhere in the document on the way out. It builds the certificate payload from the audit trail, then makes a single call to the Python service, which composites the marks, appends the evidence page and applies the seal in one pass. The level actually achieved is stored, not the level requested.

Running underneath all of it

Every step writes a hash-chained audit event. Each row's hash covers the previous row's, and PostgreSQL rejects UPDATE and DELETE outright. That chain is printed into the Certificate of Completion, which then goes inside the sealed, timestamped PDF - so the record of what happened and the document itself end up cross-witnessing each other.

7. Question three: competition and cost

List prices verified August 2026. Annual billing unless stated. Amounts in USD unless marked INR.

Global

Product	Price	Notes
DocuSign	Personal $10/mo ($15 monthly, 5 envelopes/mo); Standard $25/user/mo ($45 monthly); Business Pro $40/user/mo ($65 monthly)	Annual plans capped near 100 envelopes per user per year. SMS delivery from $0.40 per send, ID verification from $2.50 per attempt
Adobe Acrobat Sign	Standard Individual $12.99/user/mo; Pro Individual $19.99; Standard Teams $14.99; Pro Teams $23.99	Standard team capped near 150 transactions per user per year; monthly billing around 50 percent premium
Dropbox Sign	From about $15/user/mo; free tier of 3 documents a month	API plans from about $100/mo
PandaDoc	Essentials about $19/seat/mo; Business about $49/seat/mo	Document-generation focused
SignNow	From about $8/user/mo	Cheapest mainstream option
Zoho Sign	Free tier; paid from about $10/user/mo	

India

Structurally different - priced per transaction rather than per seat. That matters a great deal if signing is embedded in a product rather than used by a back-office team.

Product	Price
Leegality	Licence-free Basic tier: non-Aadhaar eSign about INR 15 per signatory, Aadhaar eSign about INR 25, e-stamping from INR 45 per paper
Digio	Contact sales; per transaction
SignDesk	Contact sales; per transaction
eMudhra emSigner / NSDL-Protean	CCA-licensed ESPs, per eSign transaction
Market range	INR 3 to INR 25 per signature, volume-dependent

Open source - the real alternative to building from scratch

Product	Price
DocuSeal (AGPLv3)	Self-host free; managed hosting from about EUR 9 per month
Documenso (AGPLv3)	Self-host free; cloud from about $30/mo; has PAdES support
OpenSign (AGPLv3)	Self-host free; cloud from about $30/mo

Break-even

Ten administrators sending around 500 envelopes a month:

	Annual
DocuSign Business Pro, 10 seats	$4,800 and above, before envelope overages

	Annual
In-house running cost	About $720 compute, $60 storage, $12 email and $300 to $500 for a document-signing certificate - roughly $1,100

That looks like a clear win until the build is priced in: roughly 120 to 200 hours of engineering, plus ongoing patching, certificate renewal, timestamp-authority monitoring and backup verification. Realistic break-even is **twelve to twenty-four months**.

The honest conclusion: build in-house when signing is a feature of your product - embedded, white-labelled, per-transaction economics, deeply integrated with your own workflow and identity system. Buy when it is an internal back-office need. This project is worth building because it is the former; for a team that simply needs contracts signed, DocuSign is cheaper than the engineer-months.

8. Question four: security of building in-house versus buying

Real advantages of building it

- **Data residency and minimisation.** Contracts never leave your own network. Directly relevant to India's DPDP Act 2023 and GDPR Article 44 transfer rules: you choose the region, the vendor chooses theirs.
- **Blast radius.** An e-signature SaaS is a high-value aggregated target - one breach exposes every customer's contracts at once. A single-tenant instance is far less attractive and its compromise is not systemic.
- **Key custody.** The signing key lives in your own KMS or HSM. No third party can be compelled, phished or tricked into signing on your behalf.
- **Evidence ownership.** The audit trail is in your database in a format you control and can export. If a vendor account lapses, their audit trails go with it - precisely when litigation makes you want them.
- **Access-control fidelity.** Signing permissions ride your existing roles and SSO. No per-seat pricing quietly pushing teams into account-sharing, which is a real security anti-pattern created by a commercial model.
- **No third-party code in the ceremony.** Vendor signing pages load analytics and CDN assets; a supply-chain compromise there sits directly inside the signing flow. This build serves its own fonts for that reason.
- **Deletion is real.** You can guarantee hard deletion and a defined retention policy - and here it is enforced by a scheduled command, not merely described in a privacy notice.
- **No secondary use.** Your documents are not subject to a vendor's terms covering analytics or model training.

The counterweights, which matter just as much

1. You inherit patching, dependency vulnerabilities, certificate lifecycle, timestamp-authority availability, key rotation and backup-integrity testing. Permanently.
2. You have no SOC 2 Type II, ISO 27001, HIPAA BAA or 21 CFR Part 11 attestation. Enterprise buyers ask for these, and saying you built it yourself is not an answer.
3. In a dispute, DocuSign's audit trail carries two decades of precedent and available expert witnesses. Yours is novel, and you must be prepared to prove your own process rather than point at an established one.
4. **In India, a legally recognised electronic signature under IT Act section 3A requires an eSign service from a CCA-licensed ESP bound to Aadhaar or PAN.** No amount of engineering substitutes for that licence.

The position this system takes

A hybrid. Own the interface, the storage, the workflow and the evidence; delegate identity binding and certificate issuance to a licensed authority. The code reflects this: a sealer seam lets the same envelope flow route either to the in-house PAdES sealer or to a CCA-licensed ESP, without touching anything above it.

9. Question five: core concepts, and what a signature captures

An electronic signature is not a digital signature

An **electronic signature** is a legal concept: any electronic symbol or process attached to a record and executed with intent to sign. A typed name qualifies. A **digital signature** is a cryptographic mechanism binding a key to specific document bytes. The second is the technology commonly used to make the first trustworthy, but they are not the same thing, and conflating them is the most common mistake in this area.

Assurance tiers are jurisdiction-specific. Under eIDAS in the EU: simple, advanced, then qualified. Under India's IT Act 2000: section 3 covers a digital signature made with a certificate from a CCA-licensed authority, while section 3A covers electronic signatures listed in the Second Schedule, such as Aadhaar eSign. The US ESIGN Act and UETA are technology-neutral with no tiers at all - validity rests on intent, consent, attribution and record retention.

A consequence worth stating plainly: **in the US the evidence, not the cryptography, is what makes a signature enforceable.** That is why the audit trail in this system is treated as a first-class artefact rather than a log file.

What actually happens cryptographically

1. Define the byte range: every byte of the PDF except the gap reserved for the signature itself.
2. Hash that range with SHA-256.
3. Sign the digest with the signer's private key.
4. Wrap it in a CMS or PKCS#7 SignedData structure carrying the certificate chain and signed attributes.
5. Write it into the reserved gap as an incremental update, so every earlier revision of the document survives intact inside the same file.
6. A verifier recomputes the hash over the byte range and checks it against the signature. Any changed byte breaks it.
7. Trust comes separately, from the certificate chaining to a trusted root, with revocation checked by OCSP or CRL.

A signature cannot sign itself: embedding it would change the bytes it just hashed. That is why the PDF reserves a hole and a ByteRange array declares which spans are covered.

Why timestamps and long-term validation matter

The signer's own clock proves nothing - it can be set to anything. An RFC 3161 timestamp authority countersigns the hash using a clock nobody in the transaction controls. Certificates also expire and are revoked, so PAdES defines escalating levels:

Level	Adds	Answers
B-B	PKCS#7 over the byte range	Has this been altered? Who held the key?
B-T	RFC 3161 timestamp	When was it signed, by an independent clock?
B-LT	Certificate chain and revocation data in a DSS dictionary	Still verifiable once the authority's endpoints are gone
B-LTA	A renewable chain of document timestamps	Still verifiable decades later, after the signing certificate expires

This system produces B-LTA. A practical finding worth recording: B-LT and B-LTA require embeddable revocation data, and a bare self-signed certificate has no CRL distribution point, so it cannot reach those levels at all. With the CRL unreachable an otherwise perfect signature validates as INVALID; with it published, the same file validates cleanly. Publishing revocation data is not an optional extra.

What the ceremony captures - the evidence package

Category	Captured	Why it matters
Identity	Name, email, phone, IP - and how they were verified	Recording that someone was verified, without the method, is not evidence of anything
Intent	An explicit affirmative act, recorded as its own event	UETA and ESIGN turn on the signature being executed with intent to sign
Consent	The disclosure version and a hash of the exact text, with timestamp and IP	The question is never whether they consented, but to what wording
Attribution	IP, user agent, which token was used, every authentication event including failures	Failed attempts are evidence too
Timeline	Sent, delivered, opened, viewed, field completed, signed, completed - each in UTC	Establishes sequence, not just outcome
Document integrity	SHA-256 as uploaded, after signatures are applied, and after sealing	Proves the starting point as well as the end state
The artefact	The signature image, whether drawn, uploaded or typed, the font, and its page and coordinates	The mark itself is part of the record
Location (optional)	Coordinates and accuracy, with the consent decision	Signer-reported, never server-observed
Photograph (optional)	An image captured at signing, with the consent decision	Evidence of presence, explicitly not identity verification
Trusted time	An RFC 3161 token from an independent authority	Never the application server's clock
Tamper evidence	A hash-chained audit log and the PDF's own seal	The record and the document are separately verifiable
Certificate of Completion	A human-readable PDF appended to the document	In practice this is the page that gets read in a dispute

On the limits of a hash chain

Each event's hash covers the previous event's, so editing, deleting or reordering any event invalidates every hash that follows. A scheduled command recomputes the whole chain, and PostgreSQL triggers reject UPDATE and DELETE outright.

It is worth being precise about what that achieves. A hash chain makes tampering **detectable**, not **impossible** - an attacker with write access could rebuild the chain from the point of the edit onward. What closes that gap is that the chain's contents end up inside a PAdES-signed PDF, timestamped by an authority outside your control. A rewritten chain can no longer be made to match the sealed document, and the timestamp cannot be backdated. Neither mechanism is sufficient alone; together they are strong.

10. Security controls implemented

- Signing tokens are 256-bit and stored only as SHA-256, so a database dump yields no working links. Tokens are burned on completion.
- A one-time passcode is required before the document is served. Passcodes are bcrypt-hashed, expire in ten minutes, and lock out after five failures.
- Rate limits are layered: per-IP throttles blunt automation, while the limits that protect a specific signer are per-recipient. Per-IP limits are deliberately loose because everyone behind one office network shares a counter.
- Every field is scoped to its recipient in the query itself, so another signer's field identifier resolves to nothing. Covered by an explicit test.
- Uploaded images are decoded and re-encoded server-side, stripping EXIF - including the GPS tags phone cameras write. PDFs are validated by a real parser, not by file extension or declared content type.
- The audit table rejects UPDATE and DELETE at the database level, and each row's hash covers the previous row's.
- The sealing service has no public ingress and authenticates every request with a shared-secret HMAC over the exact request body.
- Login is throttled per email-and-IP pair and answers identically for an unknown account and a wrong password, so it cannot be used to enumerate users.
- Mail credentials are encrypted at rest and never returned by the API.

11. Privacy: what is captured, and what is deleted

Location and photograph are both optional, consented, and enabled per envelope. Declining is recorded as a decision, never blocks signing, and the refusal button sits beside the other one at the same size.

Photographs are sender-enabled, not always on

A face image is special-category data under GDPR Article 9 once used to identify someone, carries heightened duties under India's DPDP Act, and is actionable per violation under Illinois BIPA. Collecting it from every signer regardless of what they are signing would be indefensible; collecting it when the sender decides a particular document warrants it is a decision someone has actually made.

Nothing is called identity verification

Without a government document, a face match against it and liveness detection, a photograph establishes that someone was present and willing to be photographed - not who they are. The certificate says so in those words. Real identity verification belongs with a licensed provider such as Onfido or Persona, which is the same buy-versus-build line argued in section 8.

Retention is enforced, not merely described

A scheduled command runs daily: photographs are deleted after 90 days, coordinates after 365, both configurable. Only the sensitive artefact goes - the record that a photograph was requested and that the signer agreed or refused is kept, because that is the part with evidential value and it holds no personal data. The purge is written into the audit chain so nothing is removed silently.

One limitation stated plainly: a sealed document already delivered is beyond recall. It carries its own copy and is tamper-evident, so nothing can be removed from it. Retention here means that the system stops holding the data, not that it ceases to exist - an inherent tension, since the property that makes the evidence trustworthy is the same one that makes it unretractable.

12. Proof that it works

None of the following is self-assessment. The signature is validated by pyHanko's command-line tool, which is not part of this codebase.

Automated checks

Suite	Result
API tests (PHPUnit, against PostgreSQL)	59 passing
End-to-end signing ceremony	46 checks passing
Sealing service HTTP tests	24 checks passing
Audit chain verification	All chains verified

The API tests run against PostgreSQL rather than SQLite. The schema depends on jsonb, GIN indexes and a plpgsql trigger, and testing against another engine would skip exactly the guarantees the tests exist to prove.

Independent validation of the signature

```
pyhanko sign validate --pretty-print --trust pki/out/ca.pem sealed.pdf
```

It reports that the signature is cryptographically sound, that the timestamp is backed by a trusted authority - DigiCert SHA256 RSA4096 Timestamp Responder - and that the signature is judged **VALID**. In Adobe Acrobat Reader, after trusting the issuing certificate, the signature panel shows the document as signed with long-term validation enabled.

Tamper detection

A test seals a document, changes one byte of page content, and re-validates. The altered file still opens perfectly and its signature reports as broken - which is exactly the point. The same file through the pyHanko CLI returns **INVALID**.

13. Engineering notes

Bugs worth recording, because several were only reachable in production and each one taught something.

- **jsonb reorders object keys.** The audit hash chain broke on its first real run: a payload written in one key order came back in another, re-serialised differently, and the recomputed hash no longer matched. Fixed by canonicalising keys before hashing. There is a regression test.
- **Queued mailables cannot carry binary.** The signed-copy mailable held the raw PDF as a job property; the payload failed to JSON-encode and the signed copy silently never sent. The end-to-end suite passed anyway because it did not check delivery - so an assertion was added.
- **Laravel redirects unauthenticated guests to a login route** that does not exist in an API-plus-SPA application. Any request without an Accept: application/json header produced a 500 where a 401 belonged. Invisible in development, immediate in production where crawlers open API URLs directly.
- **nginx does not know the .mjs extension.** pdf.js loads its worker via a dynamic import, which browsers MIME-check strictly. Served as application/octet-stream the browser refuses it, and the document never renders. Rather than configure every future host, the worker is now copied and served as .js at build time.
- **A React ref is null on the line after setState.** The camera preview stayed black because the media stream was assigned before the video element had rendered. Found only by installing a fake camera and driving the real component - testing the endpoint had proved nothing about the browser path.
- **pdf.js needs a foreground tab.** Rendering is scheduled through requestAnimationFrame, which browsers suspend while the document is hidden, so the render promise never resolves in a background tab.

14. Known limitations

- **The development CA is not a public trust anchor.** Adobe shows the signature as valid only after its certificate is trusted manually. Production needs a commercial document-signing certificate, or a CCA-licensed ESP for recognition under IT Act section 3A. The sealer seam exists for exactly that swap.
- **No per-signer digital signature.** The seal is an organisational one, as with DocuSign and Adobe Sign - the signer is bound by the evidence package, not by a key they control. A signer-held key requires a CA or ESP.
- **No unit-test suite for the single-page app.** The interactive behaviour that matters needs heavy browser mocking; coverage came from the end-to-end run and manual walkthroughs. A real gap, not a design choice.
- **Multi-signer routing order** is enforced server-side but has not been exercised end to end.
- **Backups must include the storage directory.** With local disk, a database backup alone loses every document, signature image and photograph.

Sources

Pricing and library capabilities checked August 2026. DocuSign and Adobe Acrobat Sign pricing via Signeasy and Costbench. SignNow and Dropbox Sign via Unkoa; Zoho Sign from zoho.com. Leegality pricing via productgrowth.in; India eSign comparison via signyu.com and peko.one. Open-source alternatives via sliplane.io and eversign. pyHanko capabilities from docs.pyhanko.eu and the project repository. FPDI limitations from manuals.setasign.com.